Turbulance: A Formal Specification of a Declarative Language for Evidence-Bearing, Uncertainty-Aware Computation

Lexical Structure, Grammar, Type System, and Operational Semantics

Kundai Farai Sachikonye
Technical University of Munich
kundai.sachikonye@wzw.tum.de

June 6, 2026

Abstract

We give a complete, self-contained formal specification of *Turbulance*, a declarative programming language whose primitive notions are not values and side effects but *evidence*, *hypotheses*, and *graded uncertainty*. Turbulance unifies three computational paradigms that are usually kept apart. From the imperative and functional traditions it inherits bindings, first-class functions, and structured control flow. From logic and argumentation it inherits *propositions* carrying named *motions* that are confirmed or refuted by *support* and *contradict* statements over observed data. From probabilistic and fuzzy computation it inherits *points*—semantic units annotated with a confidence and a distribution over interpretations—together with *resolution* operators that collapse a point to a determinate value under an explicit strategy. We formalise the language in five layers. (i) A lexical grammar over a fixed token alphabet. (ii) A context-free *concrete grammar* in extended Backus–Naur form, with a fully specified operator-precedence hierarchy. (iii) An *abstract syntax* together with a desugaring translation. (iv) A *type system* that is gradual: it admits a dynamic type and a consistency relation so that annotated and unannotated code interoperate, while guaranteeing soundness on the fully annotated fragment. (v) An *operational semantics*, given in big-step form, comprising the deterministic functional core, a confidence algebra on the unit interval, an argumentation semantics for propositions, a resolution semantics for points, and rules for the four "hybrid" iteration constructs (`cycle`, `drift`, `flow`, `roll`). We then formalise the *four-file project model*, in which a program is a linked tuple of an orchestration unit, a dependency graph, a state surface, and a decision log. Finally we establish the metatheory: determinacy of evaluation, type safety (progress and preservation) for the static fragment, boundedness and monotonicity of the evidence semantics, and termination criteria for the iteration constructs. Throughout, the design is related to the standard literature on formal languages, type systems, operational semantics, probabilistic and fuzzy computation, and abstract argumentation. The specification is intended to be precise enough to serve as an implementation blueprint and stable enough to serve as a language standard.

Keywords: programming-language specification, operational semantics, gradual typing, probabilistic programming, fuzzy logic, abstract argumentation, evidence theory, declarative languages, domain-specific languages.

Contents

1 Introduction

1.1 The problem: computation that reasons under evidence

Mainstream programming languages model computation as the transformation of *determinate* values: an expression denotes a value, a statement updates a store, and a function maps inputs to outputs [22, 26, 35]. This model is an excellent fit for systems whose inputs are known and whose correctness criteria are crisp. It is a poor fit for a large and growing class of tasks—scientific data analysis, evidence synthesis, diagnostic reasoning—whose inputs are *observations* of uncertain provenance and whose outputs are *conclusions* that are graded, defeasible, and accountable to the evidence that produced them. In these tasks the programmer is forced to encode, by hand and without language support, three notions that the host language does not name: a *hypothesis* under test, a body of *evidence* bearing on it, and a *degree of belief* attached to every intermediate result. The encoding is repetitive, error-prone, and opaque; uncertainty is threaded through ordinary floating-point variables, and the chain of reasoning that justifies a conclusion is lost.

Turbulance is a language in which these three notions are primitive. A *proposition* is a first-class declaration of a hypothesis, decomposed into named *motions*; a motion is confirmed or refuted by *support* and *contradict* statements evaluated against data; and the confirmation of a motion is not a Boolean but a confidence drawn from the unit interval, accumulated by an explicit and inspectable argumentation procedure [11, 28, 33]. Orthogonally, a *point* is a unit of meaning—typically a fragment of text or a structured datum—that carries both a confidence and a distribution over candidate *interpretations*; a *resolution* operator collapses a point to a determinate value under an explicit *strategy* (maximum likelihood, Bayesian weighting, conservative or exploratory selection, full distribution), so that the act of disambiguation is itself a named, auditable computational step [12, 21, 37].

1.2 Design principles

The language obeys five design principles, each of which has direct consequences for the formal definition that follows.

1. **Uncertainty is first-class, not encoded.** Confidence is a semantic domain of the language (section 5), not a convention on `Float` values. Every point, motion, and resolution carries a value in $[0, 1]$ with a fixed algebra of combination (section 5.2).

2. **Hypotheses are programs.** A proposition is a declaration with operational meaning: evaluating it runs an argumentation procedure that returns a verdict and a confidence (section 7.4). This realises, at the level of language semantics, the view of inference as a defeasible debate rather than a monotone deduction [4, 11].

3. **Determinism and probability cohere in one control flow.** The same program text mixes deterministic iteration (`for`, `while`) with probabilistic iteration (`considering`, `drift`) and convergent iteration (`roll until settled`). The semantics treats these uniformly as instances of a single iteration schema parameterised by how the loop variable and its auxiliary confidence are bound (section 7.6).

4. **Types are gradual.** Turbulance is dynamically typed with optional annotations and strong inference. We model this with a dynamic type and a consistency relation in the manner of Siek and Taha [30], obtaining interoperability between annotated and unannotated code together with soundness on the annotated fragment (sections 6 and 9).

5. **A program is a four-part project.** Source is organised into four linked file roles—an orchestration unit (`.trb`), a dependency graph (`.ghd`), a state surface (`.fs`), and a decision log (`.hre`). We formalise this as a typed module composition with a linkage relation (section 8).

1.3 Contributions and structure

This document is the language standard. Its contributions are:

- a precise lexical grammar (section 2) and context-free concrete grammar with a complete precedence hierarchy (section 3);
- an abstract syntax and a desugaring translation that reduces the surface language to a small core (section 4);
- the semantic domains, including a confidence algebra on $[0, 1]$ and the structures of points, floors, and corpora (section 5);
- a gradual type system with judgements for the functional core, the scientific-reasoning layer, and the probabilistic layer (section 6);
- a big-step operational semantics for all of the above, including the argumentation semantics of propositions, the resolution semantics of points, and the four hybrid iteration constructs (section 7);
- the four-file execution model (section 8);
- and the metatheory: determinacy, type safety, evidence-semantics boundedness and monotonicity, and termination (section 9).

Section 10 gives worked examples; section 11 situates the language with respect to existing paradigms; section 12 concludes.

1.4 Metalanguage and notational conventions

We use standard mathematical metalanguage. $\mathbb{N}$, $\mathbb{Z}$, $\mathbb{R}$ are the naturals, integers, reals; $[0, 1]$ is the closed unit interval. For a set A, A^* is the set of finite sequences over A, and $\mathcal{M}_{\mathrm{f}}(A)$ the set of finite multisets over A. A finite partial map $f \colon A \rightharpoonup B$ has domain $\mathrm{dom}(f)$; $f[a \mapsto b]$ is the update; $\varnothing$ is the empty map. We write $\overline{x}$ for a finite sequence $x_1, \ldots, x_n$ whose length $|\overline{x}| = n$ is left implicit. Grammars are written in extended Backus–Naur form [2, 24]: $\langle \mathtt{n} \rangle$ is a nonterminal, `k` a terminal, $[\,\alpha\,]$ an optional phrase, $\{\,\alpha\,\}$ zero-or-more repetition, $\alpha \mid \beta$ alternation. Inference rules are written $\frac{\text{premises}}{\text{conclusion}}$ with a rule name; double bars are not used. Semantic (denotation) brackets are $[\![\cdot]\!]$. Big-step evaluation is written with $\Downarrow$.

Convention 1.1 (Faithfulness)**.** The grammar, types, and semantics in this document describe the language as a mathematical object. Where an implementation is mentioned it is only to fix the intended reading; the present text, not any implementation, is normative.

2 Lexical Structure

A Turbulance program is a finite sequence of Unicode code points. Lexical analysis partitions this sequence into a stream of *tokens* after discarding layout and comments. The lexical grammar is regular [16, 19, 32] and is given below as a set of regular definitions; tokenisation follows the maximal-munch (longest-match) rule, with keywords taking priority over identifiers.

2.1 Layout and comments

Whitespace comprises the space, horizontal tab, carriage return, and line feed. Whitespace separates tokens and is otherwise insignificant except where noted (Turbulance uses explicit block delimiters; see theorem 2.2). A *comment* begins with `//` or `#` and extends to the end of the line. Comments are lexically equivalent to whitespace.

2.2 Token classes

Definition 2.1 (Lexical alphabet)**.** The terminal alphabet Σ_{lex} consists of the token classes: *keywords*, *identifiers*, *numeric literals*, *string literals*, *boolean literals*, *operators*, and *delimiters*.

Identifiers and keywords.

$$\begin{aligned}
\langle \text{letter} \rangle &::= \texttt{A} \mid \cdots \mid \texttt{Z} \mid \texttt{a} \mid \cdots \mid \texttt{z} \mid \texttt{_} \\
\langle \text{digit} \rangle &::= \texttt{0} \mid \cdots \mid \texttt{9} \\
\langle \text{ident} \rangle &::= \langle \text{letter} \rangle \, \{ \, \langle \text{letter} \rangle \mid \langle \text{digit} \rangle \, \}
\end{aligned}$$

An ⟨ident⟩ that coincides with a keyword is lexed as that keyword. The *keywords* of Turbulance are the reserved words

`funxn item proposition motion evidence pattern support contradict inconclusive within given considering for each in while return ensure allow research cause point resolution resolve cycle drift flow roll until settled over on goal metacognitive import from as try catch finally project sources otherwise requirements with_confidence true false and or not`.

The keyword for function definition is **`funxn`** (not `function`); the keyword for binding is **`item`** (not `var` or `let`); the conditional keyword is **`given`** (not `if`). These choices are intrinsic to the language's vocabulary and are not abbreviations.

Literals.

$$\begin{aligned}
\langle \text{num} \rangle &::= \langle \text{digit} \rangle \, \{ \, \langle \text{digit} \rangle \, \} \, [\, \texttt{.} \, \langle \text{digit} \rangle \, \{ \, \langle \text{digit} \rangle \, \} \,] \\
\langle \text{string} \rangle &::= \texttt{"} \, \{ \, \langle \text{schar} \rangle \, \} \, \texttt{"} \\
\langle \text{schar} \rangle &::= \text{any code point except } \texttt{"} \text{ and newline} \mid \texttt{\textbackslash} \, \langle \text{esc} \rangle \\
\langle \text{esc} \rangle &::= \texttt{n} \mid \texttt{t} \mid \texttt{r} \mid \texttt{"} \mid \texttt{\textbackslash} \\
\langle \text{bool} \rangle &::= \textbf{\texttt{true}} \mid \textbf{\texttt{false}}
\end{aligned}$$

Numeric literals denote elements of $\mathbb{R}$ (the language does not distinguish a separate integer type at the value level; see section 5); the type system nonetheless tracks a refinement Num that an implementation may represent with machine integers when the fractional part is absent.

Operators and delimiters. The operator tokens, in decreasing binding strength, are

`.` `!` (unary) `* /` `+ -` `< > <= >=` `== !=` `&&` `||` `|` `|>` `=>` `=`.

The delimiters are `( ) { } [ ]` and the punctuators `, : ; .`. The colon `:` introduces a block (see section 3); the semicolon is an optional statement terminator.

Remark 2.2 (Block structure). Turbulance delimits blocks explicitly. A block is either a brace-delimited sequence `{ ... }` or a colon-introduced suite `: ...` terminated by a dedent or an explicit block end, at the discretion of the concrete syntax in section 3. To keep the grammar context-free and layout-insensitive, we take the brace form as primitive and treat the colon-suite form as syntactic sugar that an implementation's reader expands to braces. All semantics in this document are stated over the brace-delimited abstract syntax.

3 Concrete Grammar

We now give the context-free grammar of Turbulance [1, 6]. The grammar is presented in EBNF; section 4 maps it to abstract syntax. The start symbol is ⟨program⟩.

3.1 Programs and declarations

```
⟨program⟩    ::= { ⟨decl⟩ }
⟨decl⟩       ::= ⟨funDecl⟩ | ⟨propDecl⟩ | ⟨evidenceDecl⟩ | ⟨patternDecl⟩ | ⟨pointDecl⟩
               | ⟨goalDecl⟩ | ⟨metaDecl⟩ | ⟨importDecl⟩ | ⟨projectDecl⟩ | ⟨stmt⟩
⟨funDecl⟩    ::= funxn ⟨ident⟩ ( [ ⟨params⟩ ] ) : ⟨block⟩
⟨params⟩     ::= ⟨param⟩ { , ⟨param⟩ }
⟨param⟩      ::= ⟨ident⟩ [ : ⟨type⟩ ] [ = ⟨expr⟩ ]
⟨block⟩      ::= { { ⟨decl⟩ } }
⟨importDecl⟩ ::= import ⟨ident⟩ [ as ⟨ident⟩ ]
               | from ⟨ident⟩ import ⟨ident⟩ { , ⟨ident⟩ }
⟨projectDecl⟩ ::= project ⟨ident⟩ ⟨block⟩
```

3.2 The scientific-reasoning layer

```
⟨propDecl⟩      ::= proposition ⟨ident⟩ : ⟨propBody⟩
⟨propBody⟩      ::= { { ⟨motionDecl⟩ } { ⟨stmt⟩ } }
⟨motionDecl⟩    ::= motion ⟨ident⟩ ( ⟨string⟩ )
⟨supportStmt⟩   ::= support ⟨ident⟩ [ with_confidence ( ⟨expr⟩ ) ]
⟨contradictStmt⟩ ::= contradict ⟨ident⟩ [ with_confidence ( ⟨expr⟩ ) ]
⟨inconclusiveStmt⟩ ::= inconclusive ( ⟨string⟩ )
⟨evidenceDecl⟩  ::= evidence ⟨ident⟩ : sources : { ⟨source⟩ }
⟨source⟩        ::= - ⟨expr⟩
⟨patternDecl⟩   ::= pattern ⟨ident⟩ : ⟨patternBody⟩
⟨patternBody⟩   ::= { ⟨signature⟩ [ ⟨withinStmt⟩ ] }
⟨signature⟩     ::= signature : ⟨string⟩
⟨goalDecl⟩      ::= goal ⟨ident⟩ : ⟨goalBody⟩
⟨metaDecl⟩      ::= metacognitive ⟨ident⟩ : ⟨metaBody⟩
```

3.3 The probabilistic layer

```
⟨pointDecl⟩  ::= point ⟨ident⟩ = ⟨pointLit⟩
⟨pointLit⟩   ::= { ⟨field⟩ { , ⟨field⟩ } }
⟨field⟩      ::= ⟨ident⟩ : ⟨expr⟩

⟨resolveStmt⟩ ::= resolution ⟨ident⟩ ( [ ⟨args⟩ ] )
                | resolve ⟨expr⟩ [ ⟨ctx⟩ ]
⟨ctx⟩        ::= given context ( ⟨expr⟩ )

⟨cycleStmt⟩  ::= cycle ⟨ident⟩ over ⟨expr⟩ : ⟨block⟩
⟨driftStmt⟩  ::= drift ⟨ident⟩ in ⟨expr⟩ : ⟨block⟩
⟨flowStmt⟩   ::= flow ⟨ident⟩ on ⟨expr⟩ : ⟨block⟩
⟨rollStmt⟩   ::= roll until settled : ⟨block⟩
```

3.4 Statements and control flow

```
⟨stmt⟩ ::= ⟨itemStmt⟩ | ⟨assign⟩ | ⟨givenStmt⟩ | ⟨withinStmt⟩ | ⟨consideringStmt⟩
         | ⟨forStmt⟩ | ⟨whileStmt⟩ | ⟨returnStmt⟩ | ⟨ensureStmt⟩ | ⟨allowStmt⟩
         | ⟨researchStmt⟩ | ⟨causeStmt⟩ | ⟨supportStmt⟩ | ⟨contradictStmt⟩
         | ⟨inconclusiveStmt⟩ | ⟨resolveStmt⟩ | ⟨cycleStmt⟩ | ⟨driftStmt⟩
         | ⟨flowStmt⟩ | ⟨rollStmt⟩ | ⟨tryStmt⟩ | ⟨exprStmt⟩

⟨itemStmt⟩ ::= item ⟨ident⟩ [ : ⟨type⟩ ] = ⟨expr⟩
⟨assign⟩   ::= ⟨lvalue⟩ = ⟨expr⟩
⟨lvalue⟩   ::= ⟨ident⟩ | ⟨expr⟩ . ⟨ident⟩ | ⟨expr⟩ [ ⟨expr⟩ ]

⟨givenStmt⟩ ::= given ⟨expr⟩ : ⟨block⟩ [ given otherwise : ⟨block⟩ ]
⟨withinStmt⟩ ::= within ⟨expr⟩ [ as ⟨ident⟩ ] : ⟨block⟩
⟨consideringStmt⟩ ::= considering ⟨quant⟩ ⟨ident⟩ in ⟨expr⟩ : ⟨block⟩
⟨quant⟩    ::= all | these | item | ε
⟨forStmt⟩  ::= for each ⟨ident⟩ in ⟨expr⟩ : ⟨block⟩
⟨whileStmt⟩ ::= while ⟨expr⟩ : ⟨block⟩

⟨returnStmt⟩ ::= return [ ⟨expr⟩ ]
⟨ensureStmt⟩ ::= ensure ⟨expr⟩
⟨allowStmt⟩ ::= allow ⟨expr⟩
⟨researchStmt⟩ ::= research ⟨expr⟩
⟨causeStmt⟩ ::= cause ⟨ident⟩ = ⟨expr⟩
⟨tryStmt⟩  ::= try : ⟨block⟩ { ⟨catch⟩ } [ finally : ⟨block⟩ ]
⟨catch⟩    ::= catch [ ⟨ident⟩ ] [ as ⟨ident⟩ ] : ⟨block⟩
⟨exprStmt⟩ ::= ⟨expr⟩
```

3.5 Expressions and precedence

The expression grammar is stratified to encode operator precedence and associativity, following the standard cascade [1]. Lower in the cascade binds tighter. The pipe operators `|` and `|>` compose a value with a function, threading the left operand as the first argument of the right (cf. the functional-composition and dataflow traditions [2, 34]).

```
⟨expr⟩       ::= ⟨pipe⟩
⟨pipe⟩       ::= ⟨or⟩ { (|  |  |>) ⟨or⟩ }
⟨or⟩         ::= ⟨and⟩ { || ⟨and⟩ }
⟨and⟩        ::= ⟨equality⟩ { && ⟨equality⟩ }
⟨equality⟩   ::= ⟨comparison⟩ { (== | !=) ⟨comparison⟩ }
⟨comparison⟩ ::= ⟨term⟩ { (< | > | <= | >=) ⟨term⟩ }
⟨term⟩       ::= ⟨factor⟩ { (+ | -) ⟨factor⟩ }
⟨factor⟩     ::= ⟨unary⟩ { (* | /) ⟨unary⟩ }
⟨unary⟩      ::= (! | -) ⟨unary⟩ | ⟨call⟩
⟨call⟩       ::= ⟨primary⟩ { ⟨callSuffix⟩ }
⟨callSuffix⟩ ::= ( [ ⟨args⟩ ] ) | . ⟨ident⟩ | [ ⟨expr⟩ ]
⟨args⟩       ::= ⟨expr⟩ { , ⟨expr⟩ }
⟨primary⟩    ::= ⟨ident⟩ | ⟨num⟩ | ⟨string⟩ | ⟨bool⟩ | ( ⟨expr⟩ )
               | ⟨arrayLit⟩ | ⟨mapLit⟩
⟨arrayLit⟩   ::= [ [ ⟨args⟩ ] ]
⟨mapLit⟩     ::= { [ ⟨field⟩ { , ⟨field⟩ } ] }
```

All binary operators are left-associative except assignment (section 3 treats assignment as a statement, not an expression, so no associativity question arises). Unary operators are prefix and bind tighter than any binary operator but looser than call/index/member suffixes.

3.6 Types in the surface syntax

```
⟨type⟩ ::= Str | Num | Bool | Unit | Value
         | TextUnit | Point | Points | Entity | Floor | Corpus
         | List [ ⟨type⟩ ] | Map [ ⟨type⟩ ]
         | ⟨type⟩ => ⟨type⟩
```

Theorem 3.1 (Unambiguity of the core grammar)**.** *The grammar of section 3.5, restricted to expressions, is unambiguous, and the full statement grammar is unambiguous up to the dangling-**given** resolution that binds an **otherwise** clause to the nearest preceding **given**.*

Proof sketch. The expression stratification assigns each operator a unique precedence level and left-associativity, so every operator string has a unique parse by the standard argument for precedence-cascade grammars [1]. Within a level, the left-recursive repetition { *op* ⟨e⟩ } forces left grouping. The only statement-level ambiguity is the optional **otherwise** clause, which is resolved by the nearest-match (most-closely-nested) rule, exactly as for the dangling-else of ALGOL-style languages [24]; with that rule fixed, every sentential form has a unique derivation. The remaining productions are $LL(1)$ after left-factoring of the ⟨stmt⟩ alternatives, each of which is selected by a distinct leading keyword. □

4 Abstract Syntax and Desugaring

The concrete grammar contains redundancy for the programmer's convenience. We reduce it to an *abstract syntax* (a small core) by a structural desugaring $\mathcal{D}$. Semantics in section 7 are defined on the core.

4.1 Core abstract syntax

Definition 4.1 (Core terms)**.** The core distinguishes *expressions* e and *statements* s:

$$
\begin{aligned}
e ::= {} & n \mid \mathit{str} \mid b \mid x \mid e\ \widehat{op}\ e \mid \widehat{\neg}\, e \mid e(\overline{e}) \mid e.\ell \mid e[e] \mid [\overline{e}] \mid \{\overline{\ell : e}\} \\
& \mid \mathsf{point}\{\overline{\ell : e}\} \mid \mathsf{resolve}(e, \varsigma) \mid \mathsf{textop}(\theta, e, \overline{e}) \\
s ::= {} & \mathsf{item}\ x = e \mid x := e \mid \mathsf{seq}(\overline{s}) \mid \mathsf{given}(e, s, s) \mid \mathsf{within}(e, x, s) \\
& \mid \mathsf{foreach}(x, e, s) \mid \mathsf{while}(e, s) \mid \mathsf{return}(e) \mid \mathsf{ensure}(e) \mid \mathsf{fun}(f, \overline{x{:}\tau}, s) \\
& \mid \mathsf{prop}(P, \overline{M{:}\mathit{str}}, s) \mid \mathsf{support}(M, e) \mid \mathsf{contradict}(M, e) \mid \mathsf{inconc}(\mathit{str}) \\
& \mid \mathsf{cycle}(x, e, s) \mid \mathsf{drift}(x, e, s) \mid \mathsf{flow}(x, e, s) \mid \mathsf{roll}(s) \\
& \mid \mathsf{consider}(\kappa, x, e, s) \mid \mathsf{import}(\dot{\cdot}) \mid \mathsf{try}(s, \overline{c}, s)
\end{aligned}
$$

where x, f, P, M range over identifiers, ℓ over field labels, θ over text-operation tags, ς over resolution strategies (section 7.5), $\kappa \in \{\mathsf{all}, \mathsf{these}, \mathsf{item}\}$ over the **`considering`** quantifiers, and $\widehat{op}, \widehat{\neg}$ over the binary and unary operators of section 2.

4.2 Desugaring

The translation $\mathcal{D}$ from concrete to core is homomorphic on shared constructs; the nontrivial clauses are:

- **Suites to blocks.** A colon-suite is mapped to a brace block (theorem 2.2); a sequence of declarations/statements becomes $\mathsf{seq}(\dots)$.
- **Conditionals.** $\mathsf{given}\ e : s$ without an **`otherwise`** desugars to $\mathsf{given}(e, \mathcal{D}[\![s]\!], \mathsf{seq}())$; with an **`otherwise`** clause s', to $\mathsf{given}(e, \mathcal{D}[\![s]\!], \mathcal{D}[\![s']\!])$.
- **Considering quantifiers.** $\mathsf{considering}\ \kappa\ x\ \mathsf{in}\ e : s$ keeps κ; the default (ε) is item. The **`all`**/**`these`** forms differ semantically in whether the body filters (section 7.6).
- **Pipes.** $e_1 \mid e_2 \mapsto e_2(\mathcal{D}[\![e_1]\!])$ and $e_1 \mid\!> e_2 \mapsto e_2(\mathcal{D}[\![e_1]\!])$ when e_2 is a call target; if $e_2 = g(\overline{a})$ then the left operand is inserted as the first argument: $g(\mathcal{D}[\![e_1]\!], \overline{a})$.
- **Function defaults.** A parameter $x : \tau = e_0$ desugars to a guard at entry binding x to e_0 when the corresponding argument is absent.
- **Pattern and evidence declarations** desugar to a prop scaffold and a resource binding respectively (sections 7.4 and 8).
- **Text operators.** An application of a text-operation keyword (e.g. divide, $\mathsf{summarize}$) on a $\mathsf{TextUnit}$ desugars to $\mathsf{textop}(\theta, e, \overline{e})$.

Proposition 4.2 (Desugaring is meaning-preserving by definition)**.** *The dynamic semantics of section 7 is defined on core terms; the meaning of a concrete program p is by definition the meaning of $\mathcal{D}[\![p]\!]$. Hence $\mathcal{D}$ is trivially adequate, and any two concrete programs with equal desugaring are observationally equivalent.*

5 Semantic Domains

We fix the mathematical objects over which programs compute. The development follows the domain-theoretic tradition [27, 31, 35] but stays elementary: all domains here are sets (with order where needed), and recursion is handled operationally in section 7.

5.1 Values

Definition 5.1 (Values)**.** The set of *values* $\mathbb{V}$ is the least set closed under the following formation rules:

$$\begin{aligned} v ::= \ & \mathsf{str}(s) \mid \mathsf{num}(r) \mid \mathsf{bool}(b) \mid \mathsf{none} \mid \mathsf{list}(\overline{v}) \mid \mathsf{map}(m) \\ & \mid \mathsf{clo}(\overline{x{:}\tau}, s, \sigma) \mid \mathsf{tu}(s, m) \mid \mathsf{pt}(p) \mid \mathsf{floor}(F) \mid \mathsf{ent}(p) \mid \mathsf{corpus}(s) \end{aligned}$$

where $s \in \mathit{String}$, $r \in \mathbb{R}$, $b \in \mathbb{B} = \{\top, \bot\}$, m: *Label* $\rightharpoonup \mathbb{V}$ a finite map, σ an environment (section 5.4), p a point (section 5.3), and F a floor (section 5.3). A *text unit* $\mathsf{tu}(s, m)$ is a string with metadata; an *entity* $\mathsf{ent}(p)$ is a point that has been resolved to determinate status.

5.2 The confidence algebra

Degrees of belief are elements of the unit interval. We equip $[0, 1]$ with two binary operations that govern, respectively, the *accumulation* of independent corroborating evidence and the *attenuation* of belief by opposing evidence. These operations are the algebraic heart of the language's treatment of uncertainty and are chosen to satisfy the axioms one expects of such combination [8, 17, 28].

Definition 5.2 (Confidence algebra)**.** The *confidence algebra* is the structure $\mathcal{C} = ([0, 1], \oplus, \ominus, 0, 1, \leq)$ where $\leq$ is the usual order and, for $c, d \in [0, 1]$,

$$c \oplus d \;=\; 1 - (1 - c)(1 - d), \qquad c \ominus d \;=\; c\,(1 - d).$$

The operation $\oplus$ (*noisy-or*, or probabilistic sum) accumulates support; $\ominus$ attenuates a belief c by a counter-belief d.

Proposition 5.3 (Algebraic laws)**.** *For all $c, d, e \in [0, 1]$:*

(i) $([0, 1], \oplus, 0)$ *is a commutative monoid: $\oplus$ is associative, commutative, and has unit 0;*
(ii) $\oplus$ *is idempotent-free but* inflationary*: $c \leq c \oplus d$ and $d \leq c \oplus d$, with $c \oplus d \leq 1$;*
(iii) $\oplus$ *is monotone in each argument; 1 is absorbing ($1 \oplus d = 1$);*
(iv) $\ominus$ *is antitone in its second argument and monotone in its first; $c \ominus 0 = c$, $c \ominus 1 = 0$, and $0 \leq c \ominus d \leq c$.*

Proof. (i) Writing $\bar{c} = 1 - c$, we have $c \oplus d = 1 - \bar{c}\bar{d}$, and $\overline{c \oplus d} = \bar{c}\bar{d}$; since multiplication on $[0, 1]$ is commutative, associative, with unit 1, the map $c \mapsto \bar{c}$ is an isomorphism from $([0, 1], \oplus, 0)$ to $([0, 1], \cdot, 1)$, transporting all three laws. (ii) $c \oplus d - c = (1 - c)d \geq 0$, and $c \oplus d \leq 1$ because $\bar{c}\bar{d} \geq 0$. (iii) $\partial(c \oplus d)/\partial c = 1 - d \geq 0$; and $1 \oplus d = 1 - 0 = 1$. (iv) $\partial(c \ominus d)/\partial d = -c \leq 0$, $\partial(c \ominus d)/\partial c = 1 - d \geq 0$; the boundary values are immediate. □

Remark 5.4 (Why noisy-or). Among associative, commutative, monotone combination rules on $[0, 1]$ with unit 0 and top 1 that treat sources as independent, the noisy-or is the unique one arising from the probability of at least one of independent Bernoulli events firing [25]. It is the combination rule of the language's *maximum-likelihood* aggregation strategy (section 7.4); other strategies use other combinators, all definable from $\oplus, \ominus$, and the order.

5.3 Points, interpretations, floors, corpora

The probabilistic layer rests on the notion of a *point*: a unit of meaning carrying its own uncertainty and a distribution over readings. This is the language-level reification of a fuzzy or ambiguous datum [14, 37].

Definition 5.5 (Interpretation, point)**.** An *interpretation* is a pair $\iota = (m, p)$ with $m \in \mathit{String}$ a candidate meaning and $p \in [0, 1]$ its probability. A *point* is a tuple

$$p \;=\; (\mathit{content} \in \mathit{String},\ \mathit{conf} \in [0, 1],\ I \in \mathit{Interp}^*,\ (\ell, u) \in [0, 1]^2)$$

where $I = \overline{(m_i, p_i)}$ is a finite list of interpretations subject to the sub-stochastic constraint $\sum_i p_i \leq 1$, and (ℓ, u) with $\ell \leq u$ are semantic bounds. The residual mass $1 - \sum_i p_i$ is the probability of an *unlisted* reading.

Definition 5.6 (Interpretation entropy and ambiguity)**.** For a point p with interpretations $I = \overline{(m_i, p_i)}$ and $k = |I| \geq 1$, the *interpretation entropy* is the Shannon entropy [29] of the normalised distribution $\hat{p}_i = p_i / \sum_j p_j$,

$$H(p) \;=\; -\sum_{i=1}^{k} \hat{p}_i \log_2 \hat{p}_i,$$

and the *ambiguity* is $A(p) = H(p)/\log_2 k \in [0, 1]$ (with $A(p) = 0$ when $k = 1$). A point's *intrinsic confidence* satisfies $\mathit{conf}(p) = 1 - A(p)$ when not otherwise specified.

Definition 5.7 (Floor, corpus)**.** A *floor* is a finite map $F \colon \mathit{Key} \rightharpoonup (\mathit{Point} \times \mathbb{R}_{\geq 0})$ assigning to each key a point and a nonnegative weight; it is the language's *probabilistic collection*, an iterable dictionary of weighted points. A *corpus* is a value $\mathsf{corpus}(s)$ wrapping a body of text s to be processed by stochastic traversal. The *weight-normalised* iteration order of F visits each (k, p, w) with selection mass $w / \sum_{k'} w_{k'}$.

5.4 Environments, stores, and the trace

Definition 5.8 (Environment)**.** An *environment* $\sigma \colon \mathit{Var} \rightharpoonup \mathbb{V}$ is a finite partial map from identifiers to values. The empty environment is $\varnothing$; update is $\sigma[x \mapsto v]$; lookup is $\sigma(x)$.

Two distinguished components accompany the environment during the evaluation of a proposition or a hybrid loop and record, respectively, the state of an ongoing argument and the auxiliary confidences produced by probabilistic iteration.

Definition 5.9 (Debate state)**.** A *debate state* is a finite map $\Delta \colon \mathit{Motion} \rightharpoonup (\mathcal{M}_{\mathrm{f}}(\mathit{Ev}) \times \mathcal{M}_{\mathrm{f}}(\mathit{Ev}))$ assigning to each motion a pair $(\mathsf{Aff}, \mathsf{Con})$ of finite multisets of *evidence items*. An evidence item is a triple $\eta = (\mathit{strength} \in [0, 1], \mathit{conf} \in [0, 1], \mathit{weight} \in \mathbb{R}_{\geq 0})$.

Definition 5.10 (Configuration)**.** A *configuration* is a tuple $\langle \sigma, \Delta \rangle$. Evaluation threads a configuration; ordinary statements leave Δ unchanged, while $\mathsf{support}/\mathsf{contradict}$ statements extend it.

6 The Type System

Turbulance is dynamically typed with optional annotations and inference. We formalise this as a *gradual* type system in the sense of Siek and Taha [30]: a dynamic type $\star$ (written `Value` in source) together with a *consistency* relation $\sim$ lets annotated and unannotated code interoperate, while the fully annotated fragment enjoys the soundness theorem of section 9.

6.1 Types and their order

Definition 6.1 (Types)**.** The set of types is generated by

$$\tau ::= \mathsf{Str} \mid \mathsf{Num} \mid \mathsf{Bool} \mid \mathsf{Unit} \mid \mathsf{TextUnit} \mid \mathsf{Point} \mid \mathsf{Points} \mid \mathsf{Entity} \mid \mathsf{Floor} \mid \mathsf{Corpus} \mid \mathsf{List}[\tau] \mid \mathsf{Map}[\tau] \mid \tau \to \tau \mid \star.$$

$\star$ is the dynamic (unknown) type. We call a type *static* if $\star$ does not occur in it.

Definition 6.2 (Subtyping)**.** The subtype relation $<:$ is the least preorder containing:

$$\mathsf{Entity} <: \mathsf{Point}, \qquad \mathsf{Point} <: \mathsf{Points} \text{ is } \textit{not} \text{ assumed}, \qquad \mathsf{Points} \equiv \mathsf{List}[\mathsf{Point}],$$

together with the covariant rules $\dfrac{\tau <: \tau'}{\mathsf{List}[\tau] <: \mathsf{List}[\tau']}$, $\dfrac{\tau <: \tau'}{\mathsf{Map}[\tau] <: \mathsf{Map}[\tau']}$, and the standard contravariant/covariant rule for functions $\dfrac{\tau_1' <: \tau_1 \quad \tau_2 <: \tau_2'}{\tau_1 \to \tau_2 <: \tau_1' \to \tau_2'}$. A resolved point is a point (hence $\mathsf{Entity} <: \mathsf{Point}$); a Points value is a homogeneous list of points.

Definition 6.3 (Consistency)**.** The *consistency* relation $\sim$ is the least symmetric relation with

$$\tau \sim \tau, \qquad \star \sim \tau, \qquad \tau \sim \star,$$

closed under the same structural rules as $<:$ (covariantly on constructors, contra/co on $\to$). Consistency is *not* transitive; $\star \sim \mathsf{Num}$ and $\star \sim \mathsf{Str}$ do not give $\mathsf{Num} \sim \mathsf{Str}$.

We write $\tau \lesssim \tau'$ for "τ is consistent with a subtype of τ'": $\exists \tau''.\ \tau \sim \tau'' <: \tau'$. This is the relation used at function application and assignment.

6.2 Typing judgements: expressions

A typing context Γ: $\mathit{Var} \rightharpoonup \tau$ assigns types to variables. The expression judgement is $\Gamma \vdash e : \tau$.

$$\frac{}{\Gamma \vdash n : \mathsf{Num}}\ (\text{T-Num}) \qquad \frac{}{\Gamma \vdash \mathit{str} : \mathsf{Str}}\ (\text{T-Str}) \qquad \frac{}{\Gamma \vdash b : \mathsf{Bool}}\ (\text{T-Bool})$$

$$\frac{x \in \mathrm{dom}(\Gamma)}{\Gamma \vdash x : \Gamma(x)}\ (\text{T-Var}) \qquad \frac{}{\Gamma \vdash \mathsf{none} : \mathsf{Unit}}\ (\text{T-None})$$

$$\frac{\Gamma \vdash e_1 : \tau_1 \quad \tau_1 \lesssim \mathsf{Num} \quad \Gamma \vdash e_2 : \tau_2 \quad \tau_2 \lesssim \mathsf{Num} \quad \mathit{op} \in \{+, -, *, /\}}{\Gamma \vdash e_1\ \mathit{op}\ e_2 : \mathsf{Num}}\ (\text{T-Arith})$$

$$\frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma \vdash e_2 : \tau_2 \quad \tau_1 \sim \tau_2 \quad \mathit{op} \in \{<, >, \leq, \geq, ==, !=\}}{\Gamma \vdash e_1\ \mathit{op}\ e_2 : \mathsf{Bool}}\ (\text{T-Cmp})$$

$$\frac{\Gamma \vdash e_i : \tau_i \quad \tau_i \lesssim \mathsf{Bool}\ (i = 1, 2) \quad \mathit{op} \in \{\&\&, ||\}}{\Gamma \vdash e_1\ \mathit{op}\ e_2 : \mathsf{Bool}}\ (\text{T-Logic})$$

$$\frac{\Gamma \vdash e : \tau_f \quad \tau_f \sim (\tau_1 \to \tau_2) \quad \Gamma \vdash e_i : \tau_i' \quad \tau_i' \lesssim \tau_1\ \text{(pointwise)}}{\Gamma \vdash e(\overline{e_i}) : \tau_2}\ (\text{T-App})$$

$$\frac{\Gamma \vdash e : \tau \quad \tau \lesssim \mathsf{List}[\tau'] \quad \Gamma \vdash e' : \tau'' \quad \tau'' \lesssim \mathsf{Num}}{\Gamma \vdash e[e'] : \tau'}\ (\text{T-Idx}) \qquad \frac{\Gamma \vdash e : \tau \quad \tau \lesssim \mathsf{Map}[\tau']}{\Gamma \vdash e.\ell : \tau'}\ (\text{T-Mem})$$

$$\frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma \vdash e_2 : \tau_2 \quad \tau_2 \sim (\tau_1 \to \tau_3)}{\Gamma \vdash e_1 \mid e_2 : \tau_3}\ (\text{T-Pipe})$$

$$\frac{\Gamma \vdash e_{\mathit{content}} : \tau_c \quad \tau_c \lesssim \mathsf{Str} \quad \Gamma \vdash e_{\mathit{conf}} : \tau_k \quad \tau_k \lesssim \mathsf{Num}}{\Gamma \vdash \mathsf{point}\{\ldots\} : \mathsf{Point}}\ (\text{T-Point})$$

$$\frac{\Gamma \vdash e : \tau \quad \tau \lesssim \mathsf{Point}}{\Gamma \vdash \mathsf{resolve}(e, \varsigma) : \mathsf{Entity}}\ (\text{T-Resolve})$$

The rule T-Resolve captures the central typing fact of the probabilistic layer: *resolving a point yields an entity*—a value whose ambiguity has been discharged.

6.3 Typing judgements: statements

The statement judgement is $\Gamma \vdash s \dashv \Gamma'$: under Γ, statement s is well-typed and *produces* the extended context Γ' (binding introductions flow rightward).

$$\frac{\Gamma \vdash e : \tau' \quad \tau' \lesssim \tau}{\Gamma \vdash (\mathsf{item}\ x{:}\tau = e) \dashv \Gamma[x \mapsto \tau]}\ (\text{T-ITEM}) \qquad \frac{\Gamma \vdash e : \tau}{\Gamma \vdash (\mathsf{item}\ x = e) \dashv \Gamma[x \mapsto \tau]}\ (\text{T-ITEMDYN})$$

$$\frac{x \in \mathrm{dom}(\Gamma) \quad \Gamma \vdash e : \tau' \quad \tau' \lesssim \Gamma(x)}{\Gamma \vdash (x := e) \dashv \Gamma}\ (\text{T-ASSIGN})$$

$$\frac{\Gamma \vdash s_1 \dashv \Gamma_1 \quad \Gamma_1 \vdash s_2 \dashv \Gamma_2}{\Gamma \vdash \mathsf{seq}(s_1, s_2) \dashv \Gamma_2}\ (\text{T-SEQ})$$

$$\frac{\Gamma \vdash e : \tau \quad \tau \lesssim \mathsf{Bool} \quad \Gamma \vdash s_1 \dashv \Gamma_1 \quad \Gamma \vdash s_2 \dashv \Gamma_2}{\Gamma \vdash \mathsf{given}(e, s_1, s_2) \dashv \Gamma}\ (\text{T-GIVEN})$$

$$\frac{\Gamma \vdash e : \tau \quad \tau \lesssim \mathsf{List}[\tau'] \quad \Gamma[x \mapsto \tau'] \vdash s \dashv \Gamma'}{\Gamma \vdash \mathsf{foreach}(x, e, s) \dashv \Gamma}\ (\text{T-FOREACH})$$

$$\frac{\Gamma \vdash e : \tau \quad \tau \lesssim \mathsf{Bool} \quad \Gamma \vdash s \dashv \Gamma'}{\Gamma \vdash \mathsf{while}(e, s) \dashv \Gamma}\ (\text{T-WHILE})$$

$$\frac{\Gamma \vdash e : \tau \quad \Gamma[x \mapsto \tau] \vdash s \dashv \Gamma'}{\Gamma \vdash \mathsf{within}(e, x, s) \dashv \Gamma}\ (\text{T-WITHIN})$$

$$\frac{\Gamma \vdash e : \tau \quad \tau \lesssim \mathsf{Bool}}{\Gamma \vdash \mathsf{ensure}(e) \dashv \Gamma}\ (\text{T-ENSURE}) \qquad \frac{\Gamma \vdash e : \tau}{\Gamma \vdash \mathsf{return}(e) \dashv \Gamma}\ (\text{T-RETURN})$$

$$\frac{\Gamma[\overline{x \mapsto \tau}] \vdash s \dashv \Gamma' \quad \Gamma' \text{ returns } \tau_r}{\Gamma \vdash \mathsf{fun}(f, \overline{x{:}\tau}, s) \dashv \Gamma[f \mapsto \overline{\tau} \to \tau_r]}\ (\text{T-FUN})$$

The judgement "Γ' returns τ_r" holds when every $\mathsf{return}(e)$ on a control path through the body has $\Gamma' \vdash e : \tau_e$ with $\tau_e \lesssim \tau_r$, and τ_r is the least upper bound (in $<:$) of those τ_e, or Unit if there is no return.

6.4 Typing the scientific-reasoning layer

$$\frac{\overline{M} \text{ distinct} \quad \Gamma,\ \overline{M{:}\mathsf{Motion}} \vdash s \dashv \Gamma'}{\Gamma \vdash \mathsf{prop}(P, \overline{M{:}str}, s) \dashv \Gamma[P \mapsto \mathsf{Prop}]}\ (\text{T-PROP})$$

$$\frac{\Gamma(M) = \mathsf{Motion} \quad \Gamma \vdash e : \tau \quad \tau \lesssim \mathsf{Num}}{\Gamma \vdash \mathsf{support}(M, e) \dashv \Gamma}\ (\text{T-SUPPORT}) \qquad \frac{\Gamma(M) = \mathsf{Motion} \quad \Gamma \vdash e : \tau \quad \tau \lesssim \mathsf{Num}}{\Gamma \vdash \mathsf{contradict}(M, e) \dashv \Gamma}\ (\text{T-CONTRA})$$

A support or contradict statement is well-typed only inside the scope of a proposition that declares the named motion; the type Motion is in scope exactly within T-PROP's premise. The confidence expression, when present, must be numeric (and is clamped to $[0, 1]$ at runtime; section 7.4).

6.5 Typing the hybrid iteration constructs

$$\frac{\Gamma \vdash e : \tau \quad \tau \lesssim \mathsf{Floor} \quad \Gamma[x \mapsto \mathsf{Point}] \vdash s \dashv \Gamma'}{\Gamma \vdash \mathsf{cycle}(x, e, s) \dashv \Gamma} \ (\text{T-Cycle}) \qquad \frac{\Gamma \vdash e : \tau \quad \tau \lesssim \mathsf{Corpus} \quad \Gamma[x \mapsto \mathsf{Str}] \vdash s \dashv \Gamma'}{\Gamma \vdash \mathsf{drift}(x, e, s) \dashv \Gamma} \ (\text{T-D}$$

$$\frac{\Gamma \vdash e : \tau \quad \tau \lesssim \mathsf{Floor} \quad \Gamma[x \mapsto \mathsf{Str}] \vdash s \dashv \Gamma'}{\Gamma \vdash \mathsf{flow}(x, e, s) \dashv \Gamma} \ (\text{T-Flow}) \qquad \frac{\Gamma \vdash s \dashv \Gamma'}{\Gamma \vdash \mathsf{roll}(s) \dashv \Gamma} \ (\text{T-Roll})$$

$$\frac{\Gamma \vdash e : \tau \quad \tau \lesssim \mathsf{Corpus} \text{ or } \tau \lesssim \mathsf{List}[\tau''] \quad \Gamma[x \mapsto \tau_x] \vdash s \dashv \Gamma'}{\Gamma \vdash \mathsf{consider}(\kappa, x, e, s) \dashv \Gamma} \ (\text{T-Consider})$$

where $\tau_x = \mathsf{Str}$ when $\tau \lesssim \mathsf{Corpus}$ and $\tau_x = \tau''$ otherwise. Inside a hybrid loop the implementation binds the distinguished auxiliary variables `current_weight`, `current_confidence`, `current_mode`, `current_settled` (section 7.6); these are typed Num, Num, Str, Bool respectively in Γ' within s.

7 Operational Semantics

We give a big-step (natural) operational semantics [18, 35]. Expression evaluation has the form $\langle e, \sigma \rangle \Downarrow \langle v, \sigma' \rangle$, allowing calls to have effects on the environment; statement evaluation has the form $\langle s, \sigma, \Delta \rangle \Downarrow \langle \kappa, \sigma', \Delta' \rangle$, where the *completion* $\kappa \in \{\mathsf{nrm}\} \cup \{\mathsf{ret}\, v : v \in \mathbb{V}\}$ records whether the statement finished normally or returned. A configuration that matches no rule *goes wrong*, written **wrong**; section 9 shows the static fragment never goes wrong.

7.1 Expressions

$$\frac{}{\langle \mathit{lit}, \sigma \rangle \Downarrow \langle [\![\mathit{lit}]\!], \sigma \rangle} \ (\text{E-Lit}) \qquad \frac{x \in \mathrm{dom}(\sigma)}{\langle x, \sigma \rangle \Downarrow \langle \sigma(x), \sigma \rangle} \ (\text{E-Var})$$

$$\frac{\langle e_1, \sigma \rangle \Downarrow \langle v_1, \sigma_1 \rangle \quad \langle e_2, \sigma_1 \rangle \Downarrow \langle v_2, \sigma_2 \rangle \quad v = [\![\mathit{op}]\!](v_1, v_2)}{\langle e_1 \ \mathit{op}\ e_2, \sigma \rangle \Downarrow \langle v, \sigma_2 \rangle} \ (\text{E-Bin})$$

$$\frac{\langle e, \sigma \rangle \Downarrow \langle \mathsf{clo}(\overline{x{:}\tau}, s, \sigma_0), \sigma_1 \rangle \quad \langle \overline{e_i}, \sigma_1 \rangle \Downarrow \langle \overline{v_i}, \sigma_2 \rangle}{\langle e(\overline{e_i}), \sigma \rangle \Downarrow \langle v, \sigma_2 \rangle} \ (\text{E-App})$$

where $\langle s,\ \sigma_0[\overline{x \mapsto v_i}],\ \Delta_\perp \rangle \Downarrow \langle \mathsf{ret}\, v,\ \sigma_3,\ \Delta_\perp \rangle$, or $v = \mathsf{none}$ if completion is nrm.

$$\frac{\langle e, \sigma \rangle \Downarrow \langle \mathsf{list}(\overline{v}), \sigma_1 \rangle \quad \langle e', \sigma_1 \rangle \Downarrow \langle \mathsf{num}(i), \sigma_2 \rangle \quad 0 \le i < |\overline{v}|}{\langle e[e'], \sigma \rangle \Downarrow \langle v_{\lfloor i \rfloor}, \sigma_2 \rangle} \ (\text{E-Idx})$$

Here $[\![\mathit{op}]\!]$ denotes the strict mathematical operation on values (numeric arithmetic, comparison returning bool, logical connectives with the usual short-circuit reading for `&&`, `||`); $\Delta_\perp$ is the empty debate state used when no proposition is in scope. E-App uses call-by-value and lexical scope: the closure's captured environment σ_0 is extended with the arguments.

7.2 Statements and control flow

$$\frac{\langle e, \sigma\rangle \Downarrow \langle v, \sigma_1\rangle}{\langle \mathsf{item}\ x = e, \sigma, \Delta\rangle \Downarrow \langle \mathsf{nrm}, \sigma_1[x \mapsto v], \Delta\rangle}\ (\text{S-ITEM})$$

$$\frac{\langle s_1, \sigma, \Delta\rangle \Downarrow \langle \mathsf{nrm}, \sigma_1, \Delta_1\rangle \quad \langle s_2, \sigma_1, \Delta_1\rangle \Downarrow \langle \kappa, \sigma_2, \Delta_2\rangle}{\langle \mathsf{seq}(s_1, s_2), \sigma, \Delta\rangle \Downarrow \langle \kappa, \sigma_2, \Delta_2\rangle}\ (\text{S-SEQ1})$$

$$\frac{\langle s_1, \sigma, \Delta\rangle \Downarrow \langle \mathsf{ret}\, v, \sigma_1, \Delta_1\rangle}{\langle \mathsf{seq}(s_1, s_2), \sigma, \Delta\rangle \Downarrow \langle \mathsf{ret}\, v, \sigma_1, \Delta_1\rangle}\ (\text{S-SEQ2})$$

$$\frac{\langle e, \sigma\rangle \Downarrow \langle \mathsf{bool}(\top), \sigma_1\rangle \quad \langle s_1, \sigma_1, \Delta\rangle \Downarrow \langle \kappa, \sigma_2, \Delta_2\rangle}{\langle \mathsf{given}(e, s_1, s_2), \sigma, \Delta\rangle \Downarrow \langle \kappa, \sigma_2, \Delta_2\rangle}\ (\text{S-GIVENT})$$

$$\frac{\langle e, \sigma\rangle \Downarrow \langle \mathsf{bool}(\bot), \sigma_1\rangle \quad \langle s_2, \sigma_1, \Delta\rangle \Downarrow \langle \kappa, \sigma_2, \Delta_2\rangle}{\langle \mathsf{given}(e, s_1, s_2), \sigma, \Delta\rangle \Downarrow \langle \kappa, \sigma_2, \Delta_2\rangle}\ (\text{S-GIVENF})$$

$$\frac{\langle e, \sigma\rangle \Downarrow \langle v, \sigma_1\rangle}{\langle \mathsf{return}(e), \sigma, \Delta\rangle \Downarrow \langle \mathsf{ret}\, v, \sigma_1, \Delta\rangle}\ (\text{S-RET})$$

$$\frac{\langle e, \sigma\rangle \Downarrow \langle \mathsf{bool}(\top), \sigma_1\rangle}{\langle \mathsf{ensure}(e), \sigma, \Delta\rangle \Downarrow \langle \mathsf{nrm}, \sigma_1, \Delta\rangle}\ (\text{S-ENST}) \qquad \frac{\langle e, \sigma\rangle \Downarrow \langle \mathsf{bool}(\bot), \sigma_1\rangle}{\langle \mathsf{ensure}(e), \sigma, \Delta\rangle \Downarrow \mathbf{wrong}}\ (\text{S-ENSF})$$

The rule S-ENSF gives ensure its meaning as an assertion in the sense of Hoare [15]: a failed ensure aborts the computation. The given construct is the language's guarded command [10]; with the desugaring of section 4 a missing `otherwise` branch is the no-op seq().

Bounded and unbounded iteration. For $\mathsf{foreach}(x, e, s)$ with $\langle e, \sigma\rangle \Downarrow \langle \mathsf{list}(v_1 \cdots v_n), \sigma_1\rangle$, evaluation runs s once per element, threading the configuration and stopping early on a ret:

$$\frac{\langle e, \sigma\rangle \Downarrow \langle \mathsf{list}(\overline{v}), \sigma_1\rangle \quad \langle s, \sigma_1[x \mapsto v_1], \Delta\rangle \Downarrow \cdots \Downarrow \langle \kappa, \sigma_n, \Delta_n\rangle}{\langle \mathsf{foreach}(x, e, s), \sigma, \Delta\rangle \Downarrow \langle \kappa', \sigma_n, \Delta_n\rangle}\ (\text{S-FOR})$$

where κ' is $\mathsf{ret}\, v$ if any iteration returned (the first such, after which iteration stops) and nrm otherwise. The rule for while is the usual unfolding and need not terminate.

7.3 Within: scoped structural traversal

The $\mathsf{within}(e, x, s)$ construct evaluates e to a structured value and runs s with x bound to it, providing a scope for pattern-guarded actions; when e denotes a collection the body is run once per constituent. It is the structural counterpart of foreach specialised to semantic units (text units, points): the body typically contains given guards that fire support/contradict.

$$\frac{\langle e, \sigma\rangle \Downarrow \langle v, \sigma_1\rangle \quad \langle s, \sigma_1[x \mapsto v], \Delta\rangle \Downarrow \langle \kappa, \sigma_2, \Delta_2\rangle}{\langle \mathsf{within}(e, x, s), \sigma, \Delta\rangle \Downarrow \langle \kappa, \sigma_2, \Delta_2\rangle}\ (\text{S-WITHIN})$$

7.4 Propositions: an argumentation semantics

Evaluating a proposition runs its body in a fresh debate state, then scores each motion under a resolution strategy and aggregates to a proposition verdict. This realises inference as a defeasible argument [4, 11, 33] rather than a deduction.

$$\frac{\Delta_0 = [\,\overline{M \mapsto (\varnothing, \varnothing)}\,] \quad \langle s, \sigma, \Delta_0\rangle \Downarrow \langle \_, \sigma_1, \Delta_1\rangle}{\langle \mathsf{prop}(P, \overline{M{:}str}, s), \sigma, \Delta\rangle \Downarrow \langle \mathsf{nrm}, \sigma_1[P \mapsto \mathsf{verdict}_{\varsigma}(\Delta_1)], \Delta\rangle}\ (\text{S-PROP})$$

The body's support and contradict statements extend the *inner* debate state Δ_0:

$$\frac{\langle e,\sigma\rangle \Downarrow \langle \mathsf{num}(c),\sigma_1\rangle \quad c' = \mathrm{clip}_{[0,1]}(c) \quad \eta = (c',c',1)}{\langle \mathsf{support}(M,e),\sigma,\Delta\rangle \Downarrow \langle \mathsf{nrm},\sigma_1,\Delta[M \mathrel{+}=_{\mathsf{Aff}} \eta]\rangle}\ (\text{S-Sup})$$

$$\frac{\langle e,\sigma\rangle \Downarrow \langle \mathsf{num}(c),\sigma_1\rangle \quad c' = \mathrm{clip}_{[0,1]}(c) \quad \eta = (c',c',1)}{\langle \mathsf{contradict}(M,e),\sigma,\Delta\rangle \Downarrow \langle \mathsf{nrm},\sigma_1,\Delta[M \mathrel{+}=_{\mathsf{Con}} \eta]\rangle}\ (\text{S-Con})$$

where $\Delta[M \mathrel{+}=_{\mathsf{Aff}} \eta]$ adds η to the affirmation multiset of M (symmetrically for Con), and the default confidence (when `with_confidence` is omitted) is $c' = 1$.

Definition 7.1 (Motion score and proposition verdict)**.** Fix a strategy ς. For a motion M with $\Delta(M) = (\mathsf{Aff},\mathsf{Con})$ define the *motion score* $\mathrm{score}_\varsigma(M) \in [0,1]$. For the four basic strategies, writing $a_i = (str_i, cf_i, w_i)$ for affirmations and b_j for contentions, with $W = \sum_i w_i + \sum_j w_j$:

$$\begin{aligned}
\textsc{MaxLik}: &\quad \mathrm{score} = \mathrm{clip}_[0,1]\Big(\tfrac{1}{W}\big(\textstyle\sum_i w_i\, str_i\, cf_i - \sum_j w_j\, str_j\, cf_j\big)\Big),\\
\textsc{Bayes}: &\quad \mathrm{score} = \textstyle\prod_i \mathrm{post}(\,\cdot\,; a_i) \text{ starting from prior } \tfrac{1}{2},\ \text{updated by } a_i, b_j \text{ as likelihoods},\\
\textsc{Conserv}: &\quad \mathrm{score} = \max\big(0,\ \textsc{MaxLik} - \lambda \textstyle\sum_j w_j\, str_j\big),\\
\textsc{Explor}: &\quad \mathrm{score} = \min\big(1,\ \textsc{MaxLik} + \mu \textstyle\sum_i w_i\, str_i\big),
\end{aligned}$$

with fixed penalty/boost constants $\lambda, \mu \in [0,1]$. The *verdict* of a motion is

$$\mathrm{verdict}(M) = \begin{cases} \mathsf{Supported} & \mathrm{score}_\varsigma(M) \ge \theta_+ \\ \mathsf{Contradicted} & \mathrm{score}_\varsigma(M) \le \theta_- \\ \mathsf{Inconclusive} & \text{otherwise,} \end{cases}$$

for thresholds $0 \le \theta_- < \theta_+ \le 1$, and the *proposition verdict* $\mathrm{verdict}_\varsigma(\Delta)$ aggregates the motions: it is the pair $\big(\overline{(M, \mathrm{score}_\varsigma(M), \mathrm{verdict}(M))},\ \bar{c}\big)$ with overall confidence $\bar{c}$ = score combined across motions by the strategy's combinator ($\oplus$ for MaxLik).

Remark 7.2 (Faithfulness to a debate). Definition 7.1 is a quantitative instance of Dung's abstract argumentation [11]: affirmations and contentions are arguments, contradict is an attack, and the score is a graded acceptability under the chosen semantics. The strategy is an explicit parameter, making the basis of every conclusion inspectable.

7.5 Points and resolution

A *resolution strategy* is a function that collapses a point's interpretation distribution to a determinate outcome [21, 23]. We give the canonical strategies; each maps a point p with normalised interpretations $\overline{(m_i, \hat{p}_i)}$ to a *resolution result*.

Definition 7.3 (Resolution results and strategies)**.** A *resolution result* is one of

$$\mathsf{Certain}(v) \ \mid\ \mathsf{Uncertain}(\overline{(v_i,\hat{p}_i)}, (\ell,u), \bar{c}) \ \mid\ \mathsf{Fuzzy}(\phi, \bar{t}, \sigma),$$

where ϕ is a membership function $\mathbb{V} \to [0,1]$, $\bar{t}$ its central tendency, σ its spread. For a fixed acceptance threshold θ:

$$\begin{aligned}
\textsc{MaxLik}(p) &= \begin{cases} \mathsf{Certain}(m_{i^*}) & \hat{p}_{i^*} \ge \theta,\ i^* = \arg\max_i \hat{p}_i, \\ \mathsf{Uncertain}(\overline{(m_i,\hat{p}_i)}, \mathrm{CI}, 1 - A(p)) & \text{otherwise;} \end{cases}\\
\textsc{Bayes}(p) &= \textsc{MaxLik} \text{ on the posterior } \hat{p}_i \propto \hat{p}_i \cdot \mathrm{prior}(m_i);\\
\textsc{Conserv}(p) &= \mathsf{Certain}(m_{i^\circ}), \quad i^\circ = \arg\min_i \mathrm{risk}(m_i) \text{ (safest reading)};\\
\textsc{Explor}(p) &= \mathsf{Certain}(m_{i^\bullet}), \quad i^\bullet = \arg\max_i \mathrm{info}(m_i) \text{ (most informative)};\\
\textsc{Full}(p) &= \mathsf{Uncertain}(\overline{(m_i,\hat{p}_i)}, \mathrm{CI}, 1 - A(p)).
\end{aligned}$$

$$\frac{\langle e,\sigma\rangle \Downarrow \langle \mathsf{pt}(p),\sigma_1\rangle \quad \mathsf{res} = \varsigma(p) \quad v = \mathsf{ent}(\widehat{p})}{\langle \mathsf{resolve}(e,\varsigma),\sigma,\Delta\rangle \Downarrow \langle \mathsf{nrm},\sigma_1[\_ \mapsto v],\Delta\rangle}\ (\text{S-RESOLVE})$$

where $\widehat{p}$ is p with its interpretation set collapsed according to res (a singleton for $\mathsf{Certain}$, the full set otherwise) and its confidence updated to the result's $\bar{c}$. Resolving an already-determinate entity is the identity.

7.6 The four hybrid iteration constructs

The four constructs cycle, drift, flow, roll are instances of one schema: bind a loop variable and a set of auxiliary confidences, run the body, accumulate. They differ in the *source* they traverse and in which auxiliaries they expose. We write $\sigma \Downarrow$ to abbreviate threading through the body.

Cycle (deterministic traversal of a floor). $\mathsf{cycle}(x,e,s)$ evaluates e to a floor F and runs s once per entry in weight-normalised order, binding x to the point's content and `current_weight` to the entry weight.

$$\frac{\langle e,\sigma\rangle \Downarrow \langle \mathsf{floor}(F),\sigma_1\rangle \quad \overline{(k_t,p_t,w_t)} = \mathrm{iter}(F) \quad \langle s,\sigma_1[x \mapsto \mathsf{str}(\mathit{content}(p_t)),\texttt{current\_weight} \mapsto \mathsf{num}(w_t)],\Delta\rangle \Downarrow \cdots}{\langle \mathsf{cycle}(x,e,s),\sigma,\Delta\rangle \Downarrow \langle \mathsf{nrm},\sigma_n,\Delta_n\rangle}$$

Drift (stochastic traversal of a corpus). $\mathsf{drift}(x,e,s)$ evaluates e to a corpus, draws a sequence of text fragments by the implementation's sampling procedure, and runs s per fragment, binding x to the fragment, `current_confidence` to the fragment's running confidence, and `current_mode` to the processing mode. Sampling is the only source of nondeterminism in the language; we model it as an oracle $\mathcal{O}$ that the judgement is parameterised by, so that $\langle s,\dots\rangle \Downarrow_{\mathcal{O}} \langle\dots\rangle$ is deterministic given $\mathcal{O}$ (cf. theorem 9.1).

$$\frac{\langle e,\sigma\rangle \Downarrow \langle \mathsf{corpus}(s_0),\sigma_1\rangle \quad \overline{f_t} = \mathcal{O}(s_0) \quad \langle s,\sigma_1[x \mapsto \mathsf{str}(f_t),\texttt{current\_confidence} \mapsto \mathsf{num}(c_t)],\Delta\rangle \Downarrow \cdots}{\langle \mathsf{drift}(x,e,s),\sigma,\Delta\rangle \Downarrow_{\mathcal{O}} \langle \mathsf{nrm},\sigma_n,\Delta_n\rangle}\ (\text{S-DRIFT}$$

Flow (streaming over the lines of a source). $\mathsf{flow}(x,e,s)$ treats its source as a sequence of lines, running s per line and binding `current_settled` to whether the line's processing has converged. It is deterministic.

Roll (iterate to a fixpoint). $\mathsf{roll}(s)$ repeats s until a convergence predicate holds, binding `settled`, `final_confidence`, and `iterations`. Let Φ be the per-iteration update on the ambient confidence $c \in [0,1]$ induced by s (a $\mathsf{resolve}$/guess step), and let the convergence predicate be $\mathrm{settled}(c) \equiv c \geq \theta_{\mathrm{conv}}$ or a small-change criterion $|c' - c| < \epsilon$.

$$\frac{c_0 = \sigma(\texttt{current\_confidence}) \quad c_{t+1} = \Phi(c_t) \quad N = \min\{t : \mathrm{settled}(c_t)\}}{\langle \mathsf{roll}(s),\sigma,\Delta\rangle \Downarrow \langle \mathsf{nrm},\sigma[\texttt{settled} \mapsto \mathsf{bool}(\top),\texttt{final\_confidence} \mapsto \mathsf{num}(c_N),\texttt{iterations} \mapsto \mathsf{num}(N)],\Delta\rangle}\ (\text{S-F}$$

If no such N exists the loop diverges (section 9 gives a sufficient condition for N to be finite).

Considering (probabilistic guarded iteration). $\mathsf{consider}(\kappa,x,e,s)$ iterates over the constituents of e (the sentences of a corpus, or the elements of a list), binds x, and runs s under a per-item guard; with $\kappa = \mathsf{these}$ the body itself filters (running only when its internal given succeeds), with $\kappa = \mathsf{all}$ it runs on every item, and with $\kappa = \mathsf{item}$ on a single distinguished item. The guard may be a *confidence condition* $\mathrm{conf} \bowtie \theta$ with $\bowtie \in \{>, <, \mathrm{within}, \mathrm{outside}\}$, evaluated against the running resolution confidence.

8 The Four-File Project Model

A Turbulance *project* is not a single source file but a linked tuple of four roles. We formalise the roles as typed modules and the project as their composition under a linkage relation; this is a module system in the sense of Cardelli and Wegner [5] specialised to four fixed roles.

Definition 8.1 (Project)**.** A *project* is a tuple $\Pi = (\mathcal{T}, \mathcal{G}, \mathcal{F}, \mathcal{H})$ where:

- $\mathcal{T}$ (the *orchestration unit*, file role `.trb`) is a core program (theorem 4.1); it holds the functions, propositions, and the control flow that drives the computation.
- $\mathcal{G}$ (the *dependency graph*, file role `.ghd`) is a finite labelled directed graph (V, E, ρ) whose vertices are named resources and whose labelling ρ binds each resource to an external source (a dataset, a service, a knowledge base); it is the project's import surface.
- $\mathcal{F}$ (the *state surface*, file role `.fs`) is a typed record of observable state and its presentation: a map from state keys to $\mathbb{V}$ together with a layout descriptor; it is the read-model the computation publishes.
- $\mathcal{H}$ (the *decision log*, file role `.hre`) is an append-only sequence of *decision records*; a decision record is a tuple $(\mathit{time}, \mathit{decision}, \mathit{reasoning}, \mathit{confidence} \in [0,1], \mathit{evidence\text{-}refs})$. It is the project's audit trail.

Definition 8.2 (Linkage)**.** The four roles are linked by a relation $\ell \subseteq \mathrm{dom}(\mathcal{T}) \times (\mathrm{dom}(\mathcal{G}) \cup \mathrm{dom}(\mathcal{F}) \cup \mathrm{dom}(\mathcal{H}))$ that resolves: (i) each free resource identifier of $\mathcal{T}$ to a vertex of $\mathcal{G}$; (ii) each state-publishing operation of $\mathcal{T}$ to a key of $\mathcal{F}$; and (iii) each proposition verdict and resolution of $\mathcal{T}$ to an appended record of $\mathcal{H}$. A project is *well-linked* if ℓ is total on the free identifiers and publishing/logging sites of $\mathcal{T}$.

Definition 8.3 (System state)**.** During execution the project maintains a *system state* $\Sigma = (\sigma, \Delta, \mathcal{H}^+, \mathcal{F}^+)$ extending the configuration of section 5.4 with the current decision log $\mathcal{H}^+$ and published state $\mathcal{F}^+$. Evaluation of $\mathcal{T}$ proceeds by the rules of section 7 with two additional effects: a proposition verdict (rule S-Prop) appends a record to $\mathcal{H}^+$, and an explicit publish updates $\mathcal{F}^+$.

Proposition 8.4 (Separation of effects)**.** *For a well-linked project, the orchestration semantics is conservative over the core semantics of section 7: erasing $\mathcal{H}^+$ and $\mathcal{F}^+$ from every configuration yields exactly the core evaluation of $\mathcal{T}$. Hence the decision log and state surface are* observational *—they record but do not alter the computation.*

Proof. The two additional effects (append to $\mathcal{H}^+$, update $\mathcal{F}^+$) are write-only with respect to the core: no core rule reads $\mathcal{H}^+$ or $\mathcal{F}^+$. Therefore the projection that drops these components commutes with every rule, and the diagram of core rules is recovered exactly. □

9 Metatheory

We establish the basic properties a language standard must guarantee: determinacy, type safety on the static fragment, soundness of the evidence semantics, and termination criteria.

9.1 Determinacy

Theorem 9.1 (Determinacy)**.** *The deterministic fragment of Turbulance (all constructs except* drift *and* consider *over a corpus) has a deterministic big-step semantics: if $\langle s, \sigma, \Delta\rangle \Downarrow \langle \kappa_1, \sigma_1, \Delta_1\rangle$ and $\langle s, \sigma, \Delta\rangle \Downarrow \langle \kappa_2, \sigma_2, \Delta_2\rangle$ then $(\kappa_1, \sigma_1, \Delta_1) = (\kappa_2, \sigma_2, \Delta_2)$. The full language is deterministic relative to a fixed sampling oracle $\mathcal{O}$.*

Proof. By induction on the height of the first derivation. For each syntactic form exactly one rule applies, and that rule's premises are evaluations of strictly smaller subterms (for expressions) or of the same statement on a strictly smaller remaining structure (for the bounded loops), so the

inductive hypothesis gives uniqueness of each premise's result; the conclusion is a function of those results. The only branching is between value-discriminating rule pairs (S-GivenT/S-GivenF, S-EnsT/S-EnsF, S-Seq1/S-Seq2), and these are mutually exclusive because a value is exactly one of $\mathsf{bool}(\top)$, $\mathsf{bool}(\bot)$, etc., and a completion is exactly one of nrm, $\mathsf{ret}\,v$. The constructs drift and corpus-$\mathsf{consider}$ consult $\mathcal{O}$; fixing $\mathcal{O}$ removes the only choice, and the same induction applies. □

9.2 Type safety

We prove safety for the *static fragment*: programs all of whose annotations are static types (no $\star$) and in which every position that the rules require to be consistent is in fact related by $<:$. The argument is the syntactic one of Pierce [26], Wright and Felleisen [36]. We first relate runtime values to types.

Definition 9.2 (Value typing)**.** $\vdash v : \tau$ holds when: $\vdash \mathsf{num}(r) : \mathsf{Num}$; $\vdash \mathsf{str}(s) : \mathsf{Str}$; $\vdash \mathsf{bool}(b) : \mathsf{Bool}$; $\vdash \mathsf{none} : \mathsf{Unit}$; $\vdash \mathsf{list}(\overline{v}) : \mathsf{List}[\tau]$ if each $\vdash v_i : \tau$; $\vdash \mathsf{pt}(p) : \mathsf{Point}$; $\vdash \mathsf{ent}(p) : \mathsf{Entity}$; $\vdash \mathsf{floor}(F) : \mathsf{Floor}$; $\vdash \mathsf{corpus}(s) : \mathsf{Corpus}$; $\vdash \mathsf{clo}(\overline{x{:}\tau}, s, \sigma) : \overline{\tau} \to \tau_r$ if the closure body checks at τ_r under a context typing σ. An environment is well-typed, $\vdash \sigma : \Gamma$, if $\vdash \sigma(x) : \Gamma(x)$ for all $x \in \mathrm{dom}(\Gamma)$.

Lemma 9.3 (Canonical forms)**.** *If* $\vdash v : \mathsf{Num}$ *then* $v = \mathsf{num}(r)$*; if* $\vdash v : \mathsf{Bool}$ *then* $v = \mathsf{bool}(b)$*; if* $\vdash v : \mathsf{List}[\tau]$ *then* $v = \mathsf{list}(\overline{w})$ *with each* $\vdash w_i : \tau$*; if* $\vdash v : \tau_1 \to \tau_2$ *then* v *is a closure; if* $\vdash v : \mathsf{Point}$ *then* $v \in \{\mathsf{pt}(p), \mathsf{ent}(p)\}$*; if* $\vdash v : \mathsf{Floor}$ *then* $v = \mathsf{floor}(F)$*; if* $\vdash v : \mathsf{Corpus}$ *then* $v = \mathsf{corpus}(s)$*.*

Proof. Immediate from the definition of value typing, which assigns each base/structured type a unique value former, and from $\mathsf{Entity} <: \mathsf{Point}$. □

Theorem 9.4 (Preservation)**.** *In the static fragment, if* $\Gamma \vdash e : \tau$ *and* $\vdash \sigma : \Gamma$ *and* $\langle e, \sigma\rangle \Downarrow \langle v, \sigma'\rangle$*, then* $\vdash v : \tau$ *and* $\vdash \sigma' : \Gamma$*. Likewise if* $\Gamma \vdash s \dashv \Gamma'$*,* $\vdash \sigma : \Gamma$*, and* $\langle s, \sigma, \Delta\rangle \Downarrow \langle \kappa, \sigma'', \Delta''\rangle$*, then* $\vdash \sigma'' : \Gamma'$*, and if* $\kappa = \mathsf{ret}\,v$ *then* $\vdash v : \tau_r$ *for the body's return type.*

Proof sketch. By induction on the evaluation derivation, inverting the typing derivation at each step. The base cases (E-Lit, E-Var) are immediate, the latter from $\vdash \sigma : \Gamma$. For E-Bin with an arithmetic operator, the typing rule T-Arith forces both operands to be Num in the static fragment; the IH gives $\vdash v_i : \mathsf{Num}$, hence by theorem 9.3 both are $\mathsf{num}(\cdot)$, $[\![op]\!]$ is defined and returns a num, of type Num. For E-App, T-App in the static fragment gives $\tau_f = \overline{\tau_1} \to \tau_2$ and arguments at $<: \overline{\tau_1}$; the IH and theorem 9.3 make v a closure whose body checks at τ_2; evaluating the body under the extended well-typed environment preserves τ_2 by the statement IH. The statement cases use T-Item/S-Item (the produced environment extends Γ coherently), T-Given/S-Given$\cdot$ (both branches produce Γ), and T-Seq/S-Seq$\cdot$ (threading). The scientific and probabilistic rules preserve types because S-Prop writes a Prop to P, S-Resolve writes an Entity, and S-Sup/S-Con only extend Δ, leaving Γ fixed. □

Theorem 9.5 (Progress)**.** *In the static fragment, if* $\Gamma \vdash e : \tau$ *and* $\vdash \sigma : \Gamma$ *then either* e *is a value or there is a (unique, by theorem 9.1) configuration* $\langle v, \sigma'\rangle$ *with* $\langle e, \sigma\rangle \Downarrow \langle v, \sigma'\rangle$*; in particular* e *does not go wrong. The analogous statement holds for statements, whose only non-value, non-progressing outcome would be a failed* ensure *(rule S-EnsF)—which is a* defined *abort, not a stuck state.*

Proof sketch. The potential failures are: applying a non-function, indexing a non-list, arithmetic on a non-number, or a missing variable. Each is excluded in the static fragment by the corresponding typing rule together with theorem 9.3: e.g. T-App gives the operator a function type, so by canonical forms its value is a closure and E-App applies; T-Var requires $x \in \mathrm{dom}(\Gamma)$ and $\vdash \sigma : \Gamma$ gives $x \in \mathrm{dom}(\sigma)$. The only remaining non-normal outcome is S-EnsF, which is a deliberate, defined abort in the sense of Hoare [15], not an undefined "wrong". □

Corollary 9.6 (Safety of the static fragment)**.** *A well-typed program in the static fragment never goes wrong: it either produces a value/normal completion, deliberately aborts via a failed* ensure*, or diverges.*

Remark 9.7 (The dynamic fragment). At positions typed $\star$, consistency replaces subtyping and the guarantees of theorems 9.4 and 9.5 are recovered *up to a runtime type error*: an implementation inserts a check at each $\star$-to-static boundary, and a violated check raises a defined error rather than going wrong. This is the gradual guarantee of Siek and Taha [30]: fully annotated code is sound, fully unannotated code runs with dynamic checks, and the two interoperate.

9.3 Soundness of the evidence semantics

Theorem 9.8 (Boundedness and monotonicity)**.** *For every strategy* $\varsigma \in \{\textsc{MaxLik}, \textsc{Conserv}, \textsc{Explor}, \textsc{Bayes}\}$ *and motion* M*:*

(i) $\mathrm{score}_\varsigma(M) \in [0,1]$ *(boundedness);*

(ii) adding an affirmation η *to* $\mathsf{Aff}(M)$ *does not decrease* $\mathrm{score}_{\textsc{MaxLik}}(M)$*, and adding a contention does not increase it (monotonicity);*

(iii) if $\mathsf{Con}(M) = \varnothing$ *then* $\mathrm{score}_{\textsc{MaxLik}}(M) = \bigoplus_i (str_i\, cf_i)$ *up to weight normalisation, so corroborating evidence accumulates by the noisy-or of theorem 5.2.*

Proof. (i) Each strategy clips into $[0,1]$ (Def. 7.1); Bayes returns a posterior probability, already in $[0,1]$. (ii) For MaxLik, score $= \mathrm{clip}_{[}0,1](\frac{1}{W}(S_+ - S_-))$ with $S_+ = \sum_i w_i\, str_i\, cf_i$ and S_- the analogous contention sum. Adding an affirmation increases S_+ and W; writing $f(S_+, W) = \mathrm{clip}(\frac{S_+ - S_-}{W})$, and noting each new term contributes $w\ str\ cf \le w$, the normalised score is nondecreasing because the added mass is weighted at least as favourably as the average; the contention case is symmetric. (iii) With no contentions the unclipped score is the weighted mean of $str_i\, cf_i$; under the noisy-or reading of independent corroboration (Remark after Def. 5.2) this is the $\oplus$-aggregate, which is the unique independent-source combinator (Prop. 5.3). □

Proposition 9.9 (Confidence never exceeds certainty)**.** *No sequence of* support *statements drives a motion's MaxLik score above* 1 *or a* resolve *result's confidence above* 1*. Belief is conserved within* $[0,1]$*.*

Proof. Immediate from Prop. 5.3(ii)–(iii) ($\oplus$ is bounded by 1 and 1 is absorbing) and the clipping in Def. 7.1 and Def. 7.3. □

9.4 Termination

Theorem 9.10 (Termination of bounded iteration)**.** *Evaluation of* foreach*,* cycle*,* flow*, and* consider *over a finite source terminates whenever the body terminates on each element.* while *and* roll *may diverge.*

Proof. The first four iterate over a finite list/floor/line-set of known length n; the derivation is a sequence of n body evaluations, each finite by hypothesis, so the whole is finite. while is the standard unbounded loop, and roll unfolds until a predicate holds (next theorem). □

Theorem 9.11 (Convergence of roll)**.** *Let* $\mathsf{roll}(s)$ *have per-iteration confidence update* $c_{t+1} = \Phi(c_t)$ *with* $\Phi\colon [0,1] \to [0,1]$ *continuous and either (a) a contraction (*$|\Phi(c) - \Phi(c')| \le L|c - c'|$ *with* $L < 1$*) toward a fixpoint* $c^* \ge \theta_{\mathrm{conv}}$*, or (b) monotone nondecreasing with step bounded below on* $[c_0, \theta_{\mathrm{conv}})$*. Then the convergence index* $N = \min\{t : \mathrm{settled}(c_t)\}$ *is finite and S-Roll terminates.*

Proof. (a) By the Banach fixed-point theorem $c_t \to c^*$ geometrically, so $|c_t - c^*| \le L^t |c_0 - c^*|$; choosing t with $L^t|c_0 - c^*| < \epsilon$ makes the small-change criterion fire, so $N < \infty$. (b) If Φ is monotone nondecreasing with $\Phi(c) - c \ge \delta > 0$ for $c < \theta_{\mathrm{conv}}$, then $c_t \ge c_0 + t\delta$ exceeds θ_{conv} after at most $\lceil(\theta_{\mathrm{conv}} - c_0)/\delta\rceil$ steps, so N is finite. □

Remark 9.12 (Undecidability in general). Without a hypothesis like theorem 9.11, termination of while and roll is undecidable, as for any language with general recursion [16]; the language therefore provides bounded iteration constructs (foreach, cycle) whose termination is guaranteed, and reserves unbounded iteration for cases where the programmer supplies a convergence argument.

10 Worked Examples

10.1 A proposition with motions and graded support

The following program declares a hypothesis about a dataset, decomposes it into three motions, and accumulates graded support by traversing the data with within and given guards.

```
proposition DataQuality:
    motion CompleteData("Dataset has no missing values")
    motion ConsistentFormat("All entries follow the schema")
    motion ReasonableValues("Values lie within expected ranges")

    within dataset:
        given missing_count == 0:
            support CompleteData with_confidence(0.99)
        given schema_valid:
            support ConsistentFormat with_confidence(0.95)
        given all_in_range(lo, hi):
            support ReasonableValues with_confidence(0.90)
        given outlier_rate > 0.2:
            contradict ReasonableValues with_confidence(0.6)
```

Evaluating `DataQuality` (rule S-Prop) opens an inner debate state, runs the within body once (rule S-Within), and at each satisfied guard appends an affirmation or contention (rules S-Sup/S-Con). Under MaxLik with thresholds $\theta_+ = 0.7$, $\theta_- = 0.3$ the motion `ReasonableValues` receives one affirmation $(0.90, 0.90, 1)$ and possibly one contention $(0.6, 0.6, 1)$; its score is $\mathrm{clip}_{[}0, 1](\frac{1}{2}(0.81 - 0.36)) = 0.225$ if both fire, yielding verdict Inconclusive (between θ_- and θ_+), and 0.81 if only the affirmation fires, yielding Supported. The verdict, its score, and the contributing evidence are appended to the decision log $\mathcal{H}^+$ (theorem 8.3).

10.2 A point and its resolution

```
point diagnosis = {
    content: "elevated marker with ambiguous aetiology",
    confidence: 0.7
}

item determined = resolve diagnosis given context(patient_history)
```

The literal builds $\mathsf{pt}(p)$ with $conf = 0.7$ (rule T-Point); resolve applies the ambient strategy (rule S-Resolve), producing an Entity whose interpretation set is collapsed—to a singleton if the dominant reading clears the acceptance threshold θ, otherwise to an Uncertain result carrying a confidence interval and the residual ambiguity $1 - A(p)$ (Def. 5.6, Def. 7.3). The type of `determined` is Entity (rule T-Resolve).

10.3 A hybrid loop over a probabilistic floor

```
funxn assess(floor_of_claims):
    cycle claim over floor_of_claims:
        item r = resolve claim
```

```
        given current_weight > 0.5:
            support StrongClaims with_confidence(r.confidence)
    return StrongClaims
```

`cycle` (rule S-Cycle) visits each weighted point of the floor in weight-normalised order, binding `claim` and `current_weight`; the body resolves the point and, when its selection weight is high, registers graded support. Termination is guaranteed because a floor is finite (theorem 9.10).

10.4 A four-file project, in outline

A project `biomarker` comprises: `biomarker.trb` (the orchestration unit $\mathcal{T}$ above); `biomarker.ghd` (the dependency graph $\mathcal{G}$ binding `dataset`, `patient_history`, and external knowledge bases to sources); `biomarker.fs` (the state surface $\mathcal{F}$ publishing running confidences and verdicts for display); and `biomarker.hre` (the decision log $\mathcal{H}$ receiving each verdict and resolution). By theorem 8.4 the computation's result is independent of $\mathcal{F}$ and $\mathcal{H}$, which observe but do not steer it.

11 Related Paradigms

Turbulance draws on, and can be located precisely against, several established traditions.

Functional and imperative cores. The expression language, closures, and lexical scope are standard [3, 7, 22]; the statement language with item bindings and structured control is imperative in the tradition formalised by Winskel [35]. The pipe operators are the composition combinators of functional programming [34].

Assertions and guarded commands. ensure is a Hoare-style assertion [15]; given is a guarded command [10], and the within/given idiom is a pattern-guarded structural recursion over semantic units.

Logic and argumentation. Propositions and motions echo logic programming's view of computation as proof search [20], but the semantics is *graded* and *defeasible*: it is a quantitative abstract argumentation [4, 11, 33] rather than a monotone deduction, with contradict as attack and the resolution strategy as the acceptability semantics.

Probabilistic and fuzzy computation. Points, floors, and resolution place Turbulance in the probabilistic-programming family [12, 13, 21, 23]; the confidence algebra and the membership-function results are fuzzy-set constructions [14, 37]; the accumulation of graded evidence is in the spirit of the mathematical theory of evidence [9, 28] and Bayesian confirmation [8, 17, 25], with interpretation entropy measured à la Shannon [29].

Gradual typing. The type system's dynamic type and consistency relation are exactly those of Siek and Taha [30], instantiated with the language's semantic types; subtyping follows Cardelli and Wegner [5].

The novelty is not any single ingredient but their combination in one language with a single control flow: a program may, in adjacent lines, bind a value, accumulate graded support for a hypothesis, and resolve an ambiguous datum under a named strategy—each with a precise, inspectable semantics.

12 Conclusion

We have specified Turbulance completely and from first principles: its lexical and context-free grammars, its abstract syntax and desugaring, its semantic domains (centred on a confidence algebra and the structures of points, floors, and corpora), its gradual type system, and its big-step operational semantics—including the argumentation semantics of propositions, the resolution semantics of points, and the four hybrid iteration constructs. We formalised the four-file project model as a typed module composition with a linkage relation, and proved its decision log and state surface to be purely observational. The metatheory establishes determinacy, type safety on the static fragment, boundedness and monotonicity of the evidence semantics, and termination criteria for the iteration constructs.

The specification is deliberately self-contained: every concept needed to understand the language is defined here, and the only external references are to the standard literature on which the design draws. It is intended to serve two audiences. For the implementer it is a blueprint: the grammar fixes the parser, the type rules fix the checker, and the operational rules fix the evaluator, each with stated invariants. For the language user it is a standard: the meaning of every construct is pinned down, and the guarantees—that beliefs stay within $[0, 1]$, that corroboration accumulates monotonically, that the static fragment never goes wrong, that bounded loops terminate—are theorems, not conventions.

Two extensions are left to companion documents and do not affect the core defined here: a denotational account that interprets the confidence algebra and resolution strategies in a domain of sub-probability distributions, and an operational treatment of the metacognitive and goal constructs as reflective operations over the decision log. Neither is required to read, implement, or use the language as specified above.

References

[1] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, 2 edition, 2006.

[2] John W. Backus. The syntax and semantics of the proposed international algebraic language of the zürich ACM-GAMM conference. In *Proceedings of the International Conference on Information Processing (ICIP)*, pages 125–131. UNESCO, 1959.

[3] Hendrik P. Barendregt. *The Lambda Calculus: Its Syntax and Semantics*. North-Holland, revised edition, 1984.

[4] Philippe Besnard and Anthony Hunter. *Elements of Argumentation*. MIT Press, 2008.

[5] Luca Cardelli and Peter Wegner. On understanding types, data abstraction, and polymorphism. *ACM Computing Surveys*, 17(4):471–523, 1985.

[6] Noam Chomsky. Three models for the description of language. *IRE Transactions on Information Theory*, 2(3):113–124, 1956.

[7] Alonzo Church. A formulation of the simple theory of types. *Journal of Symbolic Logic*, 5(2): 56–68, 1940.

[8] Richard T. Cox. Probability, frequency and reasonable expectation. *American Journal of Physics*, 14(1):1–13, 1946.

[9] Arthur P. Dempster. Upper and lower probabilities induced by a multivalued mapping. *The Annals of Mathematical Statistics*, 38(2):325–339, 1967.

[10] Edsger W. Dijkstra. Guarded commands, nondeterminacy and formal derivation of programs. *Communications of the ACM*, 18(8):453–457, 1975.

[11] Phan Minh Dung. On the acceptability of arguments and its fundamental role in nonmonotonic reasoning, logic programming and n-person games. *Artificial Intelligence*, 77(2):321–357, 1995.

[12] Noah D. Goodman, Vikash K. Mansinghka, Daniel M. Roy, Keith Bonawitz, and Joshua B. Tenenbaum. Church: A language for generative models. In *Proceedings of the 24th Conference on Uncertainty in Artificial Intelligence (UAI)*, pages 220–229, 2008.

[13] Andrew D. Gordon, Thomas A. Henzinger, Aditya V. Nori, and Sriram K. Rajamani. Probabilistic programming. In *Proceedings of the on Future of Software Engineering (FOSE)*, pages 167–181, 2014.

[14] Petr Hájek. *Metamathematics of Fuzzy Logic*. Kluwer Academic Publishers, 1998.

[15] C. A. R. Hoare. An axiomatic basis for computer programming. *Communications of the ACM*, 12(10):576–580, 1969.

[16] John E. Hopcroft, Rajeev Motwani, and Jeffrey D. Ullman. *Introduction to Automata Theory, Languages, and Computation*. Addison-Wesley, 3 edition, 2006.

[17] Edwin T. Jaynes. *Probability Theory: The Logic of Science*. Cambridge University Press, 2003.

[18] Gilles Kahn. Natural semantics. In *STACS 1987: 4th Annual Symposium on Theoretical Aspects of Computer Science*, volume 247 of *Lecture Notes in Computer Science*, pages 22–39. Springer, 1987.

[19] Stephen C. Kleene. Representation of events in nerve nets and finite automata. *Automata Studies, Annals of Mathematics Studies*, 34:3–41, 1956.

[20] Robert A. Kowalski. Predicate logic as programming language. In *Information Processing 74: Proceedings of IFIP Congress*, pages 569–574. North-Holland, 1974.

[21] Dexter Kozen. Semantics of probabilistic programs. *Journal of Computer and System Sciences*, 22(3):328–350, 1981.

[22] Peter J. Landin. The next 700 programming languages. *Communications of the ACM*, 9(3): 157–166, 1966.

[23] Annabelle McIver and Carroll Morgan. *Abstraction, Refinement and Proof for Probabilistic Systems*. Springer, 2005.

[24] Peter Naur et al. Revised report on the algorithmic language ALGOL 60. *Communications of the ACM*, 6(1):1–17, 1963.

[25] Judea Pearl. *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. Morgan Kaufmann, 1988.

[26] Benjamin C. Pierce. *Types and Programming Languages*. MIT Press, 2002.

[27] Dana S. Scott. Data types as lattices. *SIAM Journal on Computing*, 5(3):522–587, 1976.

[28] Glenn Shafer. *A Mathematical Theory of Evidence*. Princeton University Press, 1976.

[29] Claude E. Shannon. A mathematical theory of communication. *The Bell System Technical Journal*, 27(3):379–423, 1948.

[30] Jeremy G. Siek and Walid Taha. Gradual typing for functional languages. In *Scheme and Functional Programming Workshop*, pages 81–92, 2006.

[31] Joseph E. Stoy. *Denotational Semantics: The Scott-Strachey Approach to Programming Language Theory*. MIT Press, 1977.

[32] Ken Thompson. Programming techniques: Regular expression search algorithm. *Communications of the ACM*, 11(6):419–422, 1968.

[33] Stephen E. Toulmin. *The Uses of Argument*. Cambridge University Press, 1958.

[34] Philip Wadler. The essence of functional programming. In *Proceedings of the 19th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 1–14, 1992.

[35] Glynn Winskel. *The Formal Semantics of Programming Languages: An Introduction*. MIT Press, 1993.

[36] Andrew K. Wright and Matthias Felleisen. A syntactic approach to type soundness. *Information and Computation*, 115(1):38–94, 1994.

[37] Lotfi A. Zadeh. Fuzzy sets. *Information and Control*, 8(3):338–353, 1965.